

ADR-05 — Separación de la lógica de pagos en un servicio especializado

Estado

Propuesto

Contexto

Durante el análisis del backend del sistema **ERP Iglesias**, se identificó que las operaciones relacionadas con **pagos** pueden estar distribuidas en diferentes controladores.

En arquitecturas backend tradicionales, es común que los controladores comiencen a acumular lógica de negocio cuando gestionan múltiples responsabilidades.

Por ejemplo, controladores como:

```
EnrollmentController
OfferingController
```

pueden incluir lógica relacionada con el procesamiento de pagos.

Esto provoca que los controladores mezclen responsabilidades de **gestión de solicitudes HTTP** con **lógica de negocio**.

Problema

Cuando la lógica de pagos se encuentra distribuida en múltiples controladores:

- se duplica lógica relacionada con pagos
- los controladores se vuelven más complejos
- es difícil mantener o modificar el proceso de pagos
- se complica la integración con futuros servicios de pago externos

Esto genera **alto acoplamiento y baja cohesión en la arquitectura del backend**.

Decisión Arquitectónica

Se propone **centralizar toda la lógica de pagos en un servicio especializado** llamado:

```
PaymentService
```

Los controladores no deberán gestionar directamente la lógica de pagos, sino delegar esta responsabilidad al servicio correspondiente.

Arquitectura propuesta:

```
Controller
↓
PaymentService
↓
PaymentRepository
```

Patrón de Diseño Aplicado

Service Layer Pattern

Este patrón permite encapsular la lógica de negocio dentro de servicios especializados, evitando que los controladores gestionen directamente la lógica del sistema.

Principio SOLID Aplicado

Single Responsibility Principle (SRP)

Cada componente tendrá una responsabilidad clara:

Controller → manejar solicitudes HTTP PaymentService → gestionar la lógica de pagos PaymentRepository → acceso a datos de pagos

Cambio Concreto Propuesto

Se implementará un servicio especializado para manejar las operaciones relacionadas con pagos.

Archivo propuesto:

```
PaymentService
```

Este servicio se encargará de:

- procesar pagos

- validar operaciones de pago
- gestionar lógica relacionada con transacciones
- coordinar el acceso al repositorio de pagos

Ejemplo para Entenderlo

Antes

Los controladores manejan directamente la lógica de pagos.

```
EnrollmentController maneja pagos
OfferingController maneja pagos
```

Esto genera duplicación de lógica.

Después

La lógica de pagos se centraliza en un servicio.

```
Controllers
↓
PaymentService
↓
PaymentRepository
```

Los controladores delegan las operaciones de pago al servicio correspondiente.

Beneficios Arquitectónicos

La separación de la lógica de pagos proporciona:

- mejor organización del backend
- reducción de duplicación de código
- mayor claridad en la arquitectura del sistema
- facilidad para mantener o modificar la lógica de pagos
- preparación para integrar servicios externos de pago

Impacto en el Sistema

La funcionalidad del sistema no cambia para el usuario final.

Sin embargo, la arquitectura del backend mejora al separar correctamente las responsabilidades entre controladores, servicios y repositorios.

Resultado Arquitectónico

Arquitectura final propuesta:

```
Controller
↓
PaymentService
↓
PaymentRepository
```

Este enfoque permite mantener un backend más organizado, escalable y alineado con buenas prácticas de arquitectura de software.